

開発ロードマップ - esp32-rc-car

「スマホから操作できるカメラ付きRCカー」を、小さなステップに分けて作るための計画。各ステップは「動いた」と確認できたら次へ進む形にしている。うまくいかないときに原因を絞り込みやすくするため。

0. 全体像

フェーズ1

BLEでモーター1個を回す

← 構想 1)

Step 1-1 開発環境を整え、ESP32-C3に書き込めるようにする

Step 1-2 スマホからBLEで接続する（モーターなし）

Step 1-3 モータードライバ経由でモーター1個を回す

フェーズ2

シャーシに組んでRCカーとして走らせる

← 構想 2)

Step 2-1 シャーシを組み、モーター2個を回す（USB給電のまま）

Step 2-2 電池（18650）と電源回路を載せる

Step 2-3 床で走らせて調整する

フェーズ3

カメラ映像をスマホに表示する

← 構想 3)

Step 3-1 カメラ方式を決める（ESP32-C3ではカメラを扱えないため）

Step 3-2 カメラボード単体で映像をスマホに表示する

Step 3-3 車体に載せて、走らせながら映像を見る

Step 3-4 （任意）操作画面と映像を1画面にまとめる

現在の状況

フェーズ1・2のソフト（ファームとWebアプリ）はこのリポジトリで作成済み。2WD（左右2モーター）用だが、左モーターだけつなげば構想1)の「モーター1輪」の確認にそのまま使える。したがってフェーズ1・2は部品をそろえて配線し、動作を確かめる作業が中心になる。フェーズ3はハードもソフトも新しく作る必要がある。

フェーズ	ハード	ソフト
1. BLE＋モーター1個	未着手（部品手配から）	作成済み（ firmware/ 、 index.html ）
2. シャーシ・電池駆動	未着手	作成済み（調整のみ）
3. カメラ	未着手	未着手（Step 3-1で方式を決めてから）

使う言葉

用語	意味
BLE	Bluetooth Low Energy。少ないデータを低消費電力でやり取りする無線。操作データの送信には十分だが、映像を送るには遅すぎる
PWA	ホーム画面に追加できるWebアプリ。このリポジトリの index.html がそれに当たる

用語	意味
モータードライバ	マイコンの弱い信号で、モーターに大きな電流を流すための部品（TB6612FNG）。マイコンのピンにモーターを直接つなぐと壊れる
PWM	高速でON/OFFを繰り返して、モーターに加わる電圧を調整する方法。速度調整に使う
2S	リチウムイオン電池2本を直列にした状態（公称7.4V）
DC-DC（降圧）	電池の電圧（7.4V）をESP32用の5Vに下げる部品
MJPEG	JPEG画像を連続で送る映像形式。ESP32のカメラでよく使われる

フェーズ1：BLEでモーター1個を回す

Step 1-1 開発環境を整え、ESP32-C3に書き込めるようにする

目的: PCからESP32-C3にプログラムを書き込み、シリアルモニタで動作を確認できるようにする。

作業:

1. VS Codeに PlatformIO 拡張をインストールする
2. `esp32-rc-car.code-workspace` を開き、ESP32-C3をUSBでつなぐ
3. VS Code下のステータスバーにあるPlatformIOのボタンで、ビルド（✓）→ 書き込み（→）の順に実行する。書き込み先のポートは自動で選ばれる。書き込みもUSBケーブル1本で行い、書き込み用の機器は別にいらない
4. 同じステータスバーのシリアルモニタ（コンセントの形のボタン、115200bps）を開き、起動時のメッセージが表示されることを確認する

完了の確認:

- [] 書き込みが成功する
- [] シリアルモニタにメッセージが出る
- [] 基板上のLEDがゆっくり点滅する（接続待ち）
- [] シリアルで **B** を送ると電池電圧（USB給電中は0）が表示される（シリアルモニタの画面をクリックして **B** を打ち、Enterを押す。打った文字は画面に出ないが、ESP32が **受信: B** と返す）

つまずきやすい点:

- USBケーブルが**充電専用**だと認識されない。データ通信できるケーブルを使う
- 書き込みに失敗するときは、BOOTボタンを押しながらRESETを押して書き込みモードにする
- GPIO3が浮いていると電圧を誤判定し、低電圧で停止したままになる。分圧抵抗を付けるまでは（Step 2-1まで）GPIO3をGNDにつないでおく。Super Miniの場合、GPIO3は5V・GND・3.3Vと同じ列にあり、その列の端（USBと反対側）から数えて0→1→2→3の4番目。となりのGPIO2（起動モードを決めるピン）や3.3VをGNDにつながないよう、基板の印字で「3」を確かめてから配線する
- ピンヘッダをはんだ付けしていないと、ジャンパ線を当てても接触しない。**B** で0にならないときは接触を疑う。はんだ付けを後回しにする場合は、`firmware/src/config.h` の `VBAT_MONITOR_ENABLED` を `false` にすると監視を止められる（起動時に「注意: 電池電圧の監視を止めています」と出る）。電池を載せる前に必ず `true` に戻す

Step 1-2 スマホからBLEで接続する（モーターなし）

目的: AndroidのChromeからESP32-C3にBLEで接続し、操作データが届くことを確認する。

作業:

1. Webアプリ（[index.html](#)）を GitHub Pages で公開する（リポジトリ [sato4app/esp32-rc-car](#) の Settings → Pages で **main** ブランチを指定する）。Web BluetoothはHTTPSでないと使えないため、PCの <http://localhost> をスマホから開く方法は使えない
2. AndroidのChromeで公開したページを開き、「接続」から **ESP32-RC-Car** を選ぶ
3. ジョイスティックを動かし、「車の状態」の左・右のバーが動くことを確かめる。このバーは、ESP32が受け取った操作から計算したモーター出力を、スマホへ送り返して表示している。バーが動けば、操作データがESP32に届いている

完了の確認:

- [] スマホから接続でき、LEDが点灯に変わる
- [] 画面の「車の状態」に **USB給電** と表示される
- [] ジョイスティックを上にも倒すと左・右のバーが両方プラスに伸び、指を離すと0に戻る
- [] スマホ側で切断すると、LEDがゆっくり点滅に戻る

つまずきやすい点:

- Androidの設定でBluetoothと「付近のデバイス」（古い機種は位置情報）の許可が必要
- 一覧に出てこないときは、BLE確認アプリ「nRF Connect for Mobile」で **ESP32-RC-Car** が見えるか調べると、ESP32側とWebアプリ側のどちらの問題か切り分けられる
- ESP32-C3 Super Miniはアンテナが小さく、電波が弱い個体がある。最初は近い距離で試す

Step 1-3 モータードライバ経由でモーター1個を回す

目的: TB6612FNGを使って、スマホの操作でモーター1個を回す。

配線（ブレッドボード上。ESP32はUSB給電のまま）:

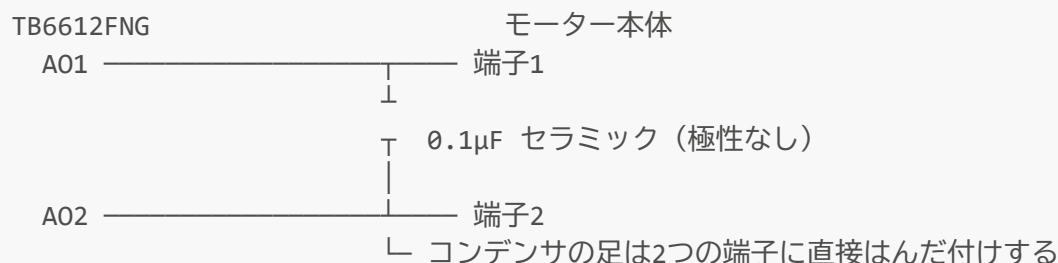


TB6612FNG	接続先
AIN1 / AIN2 / PWMA	GPIO4 / GPIO5 / GPIO6
STBY	GPIO10
VCC	ESP32の3.3V
VM	単3×4 電池ボックスの+（約6V）

TB6612FNG**接続先**

GND	ESP32のGND、電池ボックスの-（必ず共通にする）
AO1 / AO2	モーター

モーターの端子まわり（ノイズ対策の0.1μF）：



- モーターの電源はESP32（USB）から取らず、電池ボックスから取る。USBの電流ではモーターを回しきれず、ESP32がリセットする
- この段階では2S（18650）やDC-DCは使わない。扱いが簡単な単3電池で先に動作を確かめる
- モーターの端子間に0.1μFのセラミックコンデンサをはんだ付けしておく（ノイズ対策）
 - モーター背面の2枚の金属端子に、コンデンサの足を1本ずつはんだ付けする。配線（AO1/AO2へ行く線）も同じ端子に付ける
 - セラミックコンデンサには極性がないので、向きは気にしなくてよい。足はなるべく短く切る
 - ブレッドボード上（AO1/AO2の列の間）に挿しても回路としては同じだが、モーターから離れるほど効果が落ちる。はんだ付けの準備ができるまでの仮の方法にとどめる
 - 付けなくてもモーターは回る。ESP32の再起動やBLEの切断が起きる場合に効いてくる対策なので、フェーズ2で車体に組む前には必ず付ける

作業:

- 上の配線をする。モーターは車輪を付けずに、空回りの状態にする
- シリアルで **D 50 0** を送り、モーターが1秒回ることを確認する（**D -50 0** で逆回転）
- スマホから接続し、ジョイスティックで回す

完了の確認:

- ☐ シリアルの **D 50 0** / **D -50 0** で正転・逆転する
- ☐ スマホのジョイスティックで回転の速さと向きが変わる
- ☐ 指を離すと止まる
- ☐ 非常停止ボタンで止まる
- ☐ 接続中にスマホのBluetoothを切ると、約0.3秒で止まる（フェイルセーフ）

つまづきやすい点:

- まったく回らない → STBY（GPIO10）の配線、VMに電池の電圧が来ているか、GNDが共通かをテスターで確認する
- モーターが回った瞬間にESP32が再起動する → コンデンサの追加、GNDの配線の見直し
- 左右旋回を入れると、1輪だけなので片方向にしか反応しないのは正常（戦車式の配分のため）

→ ここで構想1)が完成。

フェーズ2：シャーシに組んでRCカーとして走らせる

Step 2-1 シャーシを組み、モーター2個を回す（USB給電のまま）

目的: シャーシにモーター2個と車輪を付け、左右とも正しい向きに回ることを確認する。

作業:

1. シャーシキットを組み立てる（モーター・車輪・キャスト）
2. 右モーターをTB6612FNGのB側（BIN1/BIN2/PWMB = GPIO0/1/7、BO1/BO2）につなぐ
3. 車体を持ち上げて車輪を浮かせ、シリアルで `D 50 0` を送る
4. 両輪とも「前進」の向きに回らなければ、`firmware/src/config.h` の `LEFT_MOTOR_INVERT / RIGHT_MOTOR_INVERT` を `true` にして書き込み直す

完了の確認:

- [] `D 50 0` で両輪が前進方向に回る
- [] `D 0 100` で左輪が前、右輪が後ろに回る（その場で右旋回）
- [] スマホのジョイスティックで前後・左右旋回ができる

Step 2-2 電池（18650）と電源回路を載せる

目的: USBケーブルなしで走れるように、電池と電源回路を車体に載せる。配線と部品の詳細は [hardware.md](#) の「3. 配線」「4. 電源系」にある。

作業（この順で行う）:

1. **DC-DCの出力調整:** ESP32をつなぐ前に、電池だけをつないでテスターで出力を測り、半固定抵抗で 5.0Vに合わせる
2. 電源スイッチ・電池ホルダー・DC-DC・TB6612FNGのVMを配線する
3. 470 μ Fの電解コンデンサをTB6612FNGのVM-GND間に付ける（**極性に注意**）
4. 分圧抵抗（100k Ω ・33k Ω ）を付け、GPIO3とGNDの間の仮配線を外す。`firmware/src/config.h` の `VBAT_MONITOR_ENABLED` を `false` にしていた場合は、**true に戻して**USBで書き込み直す（電源スイッチはOFFのまま。`false` のままだと過放電を防げず、電池を傷める）
5. 部品を車体に固定する（両面テープ、スペーサー、結束バンド）

完了の確認:

- [] USBをつなぐ前に電源スイッチを入れると、ESP32が起動する
- [] 起動時のシリアルモニタに「注意: 電池電圧の監視を止めています」が**出ない**
- [] スマホの「車の状態」に電池電圧（7～8.4V程度）が表示され、テスターの値とおおむね合う
- [] 車輪を浮かせた状態で、スマホからの操作で両輪が回る

つまずきやすい点・注意:

- **USBと電池（DC-DCの5V）を同時につながない。** 書き込むときは電源スイッチをOFFにする
- 電池の極性を間違えるとTB6612FNG・DC-DCが壊れる。電源を入れる前に必ず確認する
- 18650は**保護回路付き**を信頼できる販売元から買い、専用の充電器で充電する

Step 2-3 床で走らせて調整する

目的: 実際に走らせて、操作しやすいように調整する。

作業と調整項目（すべて `firmware/src/config.h`）：

気になること	調整する値
まっすぐ進まない	左右のモーターの個体差。まずは操作で補正し、気になれば後で補正値を追加する
旋回が速すぎる／遅すぎる	<code>STEER_GAIN_PERCENT</code> （今は60%）
発進が急／もっさり	<code>RAMP_PERCENT_PER_SEC</code> （今は毎秒400%）
全体に遅い	<code>MOTOR_MAX_DUTY_PERCENT</code> （今は70%。モーター定格6Vを超えないよう上げすぎない）
速すぎて操作しにくい	Webアプリの「最高速度」で制限する（ファームの変更不要）

完了の確認:

- [] 発進時にESP32が再起動しない
- [] 前進・後退・旋回が思いどおりにできる
- [] 通信が切れる距離（どこまで離れて操作できるか）を把握した
- [] 電池が減ると低電圧で止まることを確認した（6.4V未満が3秒続くと停止）

→ ここで構想2)が完成。

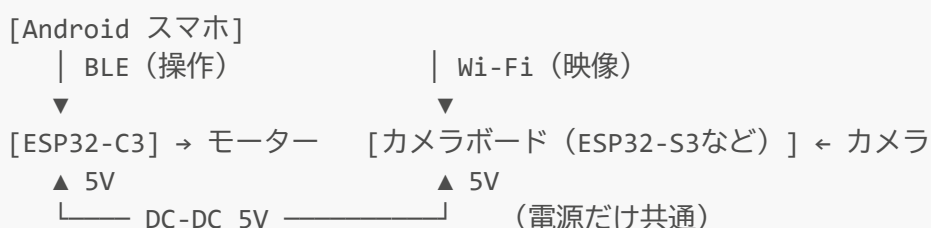
フェーズ3：カメラ映像をスマホに表示する

Step 3-1 カメラ方式を決める

フェーズ3で最初に知っておくべきことが2つある。

1. **ESP32-C3にはカメラを直接つなげない。** カメラ用の入力端子（カメラインターフェース）がなく、映像をためるメモリ（PSRAM）も足りない
2. **BLEでは映像を送れない。** BLEの実際の速度は数十kB/秒程度で、映像には足りない。映像は **Wi-Fi** で送る

そこで、**カメラ付きの別のボードを車体に追加する**方式にする。今のESP32-C3（BLEで操作・モーター制御）はそのまま残し、映像だけを別ボードがWi-Fiで送る。



- 操作と映像を別のボードにすることで、今動いている操作系に手を入れずに済む
- スマホはBLEとWi-Fiを同時に使える

カメラボードの候補:

候補	価格の目安	長所	短所
Seeed XIAO ESP32S3 Sense (推奨)	3,000～ 4,000円	小さい (約21×17mm)。USB-Cで直接書き込める。カメラ・PSRAM付き	映像送信中にかなり熱くなる
ESP32-CAM (AI-Thinker) +書き込み用ボード (ESP32-CAM-MB)	1,500～ 2,500円	安い。作例が非常に多い	書き込みに専用ボードが必要。電源が不安定だと動かない
(参考) アナログFPVカメラ+5.8GHz送信機+USB受信機	5,000円～	遅延が非常に小さい。ESP32と無関係に動く	専用アプリで表示。電波法の手続き (アマチュア無線免許など) が必要

→ 迷ったら **XIAO ESP32S3 Sense** にする。

Step 3-2 カメラボード単体で映像をスマホに表示する

目的: 車体に載せる前に、カメラボードだけで映像がスマホに映ることを確認する。

作業:

1. PlatformIOで新しいプロジェクト (例: `camera/`) を作り、カメラの作例 (`CameraWebServer`) を元に書き込む
2. カメラボードをWi-Fiのアクセスポイント (親機) にし、スマホをそのWi-Fiにつなぐ
3. スマホのChromeで `http://192.168.4.1` などを開き、映像が映ることを確認する
4. 解像度 (QVGA 320×240 ~ VGA 640×480) と画質を変えて、遅延と滑らかさを比べる

完了の確認:

- [] スマホのChromeに映像が表示される
- [] 映像の遅延の目安を把握した (手を振って、画面に映るまでの時間を見る)
- [] 数分間映し続けても止まらない (熱で止まらないか確認)

つまずきやすい点:

- ESP32のWi-Fiにつなぐと、Androidが「インターネットに接続していません」と表示し、モバイルデータ側に切り替わることがある。「このまま接続する」を選ぶ。うまくいかなければモバイルデータを切る
- 映像が止まる・再起動する → 電源不足のことが多い。USBケーブルやポートを変えてみる

Step 3-3 車体に載せて、走らせながら映像を見る

目的: カメラボードを車体の前方に載せ、走らせながら映像を見る。

作業:

1. カメラボードの5VピンをDC-DCの5V出力につなぐ（GNDも共通）
2. カメラを車体前方に固定する（L字ステー、両面テープなど）。振動で映像がぶれにくいようにする
3. スマホの画面分割（またはタブの切り替え）で、操作画面（PWA）と映像（<http://192.168.4.1>）を同時に使う

完了の確認:

- [] 電池駆動で、操作と映像が同時に使える
- [] 走行中にカメラボード・ESP32-C3のどちらも再起動しない
- [] 映像だけを見て操作できる

つまずきやすい点:

- カメラボード（Wi-Fi送信中 約0.2～0.3A）が増えるため、電源のピーク電流が増える。DC-DCは2A以上のものを使う
- BLE（2.4GHz）とWi-Fi（2.4GHz）が同じ周波数帯なので、操作が途切れやすくないか確認する

→ **ここで構想3)が、最低限の形で完成。**

Step 3-4（任意）操作画面と映像を1画面にまとめる

Step 3-3の「画面分割」で不便なら、1画面にまとめる。ただし、ブラウザの制限で**素直にはできない**。

- 操作画面（PWA）はWeb Bluetoothのため **HTTPS** で配信している
- カメラの映像は **HTTP**（<http://192.168.4.1>）
- HTTPSのページにHTTPの映像を埋め込むと、ブラウザが安全のため読み込みを止める（混在コンテンツの制限）

方式の候補（Step 3-3まで終わってから、使い勝手を見て決める）:

方式	内容	手間
A. 画面分割のまま	Step 3-3の状態。追加作業なし	なし
B. 操作もWi-Fiにまとめる	操作画面をカメラボードから配信し、操作データもWi-Fi（WebSocket）で送る。カメラボードは受け取った操作データを、UART（GPIO20/21）でESP32-C3へ渡す	中（BLEは使わなくなる）
C. Androidアプリを作る	BLEの操作と映像の表示を1つのアプリにする（Kotlin）	大（Android開発の学習が必要）

部品リスト（ステップ別）

価格は通販（秋月電子、スイッチサイエンス、Amazonなど）での**おおよその目安**。コンデンサ・抵抗などの小物は、送料を考えるとフェーズ1のときにまとめて買うのがよい。

フェーズ1で買うもの

部品	数量	価格の目安	使う場所・備考
ESP32-C3 (Super Mini または DevKitM-1)	1	500～1,500円	予備に2個あると安心
USB-Cケーブル (データ通信対応)	1	手持ち	充電専用ケーブルは不可
TB6612FNG モータードライバ モジュール	1	300～800円	ピンヘッダのはんだ付けが必要なものが多い
2WDシャーシキット (TTギアモーター×2、車輪、キャスター付き)	1	1,000～2,000円	フェーズ1ではモーター1個だけ使う
ブレッドボード	1	300～500円	
ジャンパーワイヤー (オス-オス、オス-メス)	各1セット	300～600円	
単3×4 電池ボックス (スイッチ付き) + 単3電池4本	1	300～600円	フェーズ1のモーター電源
セラミックコンデンサ 0.1μF	3～5	100円程度	モーター端子間×2、GPIO3用×1
電解コンデンサ 470μF / 16V以上	1～2	100円程度	フェーズ2で使う (まとめ買い)
抵抗 100kΩ・33kΩ	各1	数十円	フェーズ2で使う (まとめ買い)

フェーズ2で買うもの

部品	数量	価格の目安	備考
18650 リチウムイオン電池 (保護回路付き)	2	1,500～3,000円	信頼できる販売元のもの
18650 ×2 直列 電池ホルダー	1	200～500円	スイッチ付きなら電源スイッチは不要
18650対応 充電器	1	1,500～3,000円	
降圧DC-DCコンバーター (MP1584など、5V・2A以上)	1	300～800円	複数個セットが多い
電源スイッチ (3A以上)	1	100～300円	ホルダーにスイッチがなければ
電線 (AWG22～24)	適量	300円程度	モーター・電源まわり
ミニブレッドボード または ユニバーサル基板	1	200～500円	車体に載せる用

部品	数量	価格の目安	備考
スペーサー・両面テープ・結束バンド	適量	手持ち	部品の固定

フェーズ3で買うもの

部品	数量	価格の目安	備考
Seeed XIAO ESP32S3 Sense	1	3,000～4,000円	代わりにESP32-CAM+ESP32-CAM-MB（1,500～2,500円）でも可
小型ヒートシンク	1	100～300円	任意。熱対策
カメラ固定用のL字ステーなど	適量	100～300円	

工具（最初にそろえる）

工具	用途
はんだごて・はんだ・こて台	ピンヘッダ、モーターの配線、コンデンサの取り付け
テスター（デジタルマルチメーター）	電圧の確認、DC-DCの出力調整（ フェーズ2で必須 ）
ニッパー・ワイヤーストリッパー	電線の加工
精密ドライバー	シャーシの組み立て、DC-DCの半固定抵抗の調整

次にやること

- フェーズ1の部品と工具を注文する（小物はフェーズ2の分もまとめて）
- 部品が届くまでに、VS Code+PlatformIOを入れ、GitHub Pagesを設定しておく（Step 1-1・1-2の準備）
- ESP32-C3が届いたら Step 1-1 から始める